

VIETNAM NATIONAL UNIVERSITY – HO CHI MINH CITY
INTERNATIONAL UNIVERSITY
 SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

IT149IU · FUNDAMENTALS OF PROGRAMMING (PYTHON)

Lab 2: Loops and Numerical Algorithms

for / while loops, range(), lists, integer arithmetic (//, %), simple functions and formatted tables

Student name		Student ID	
Class / Lab group		Date	
Duration	150 minutes (3 periods)	Environment	Python 3.10+ · Colab / VS Code / PyCharm / Jupyter

Legend: black text = original lab content (re-formatted); red text = content added in this revision (theory, worked examples, extra exercises, setup & submission guide).

NOTE — Lab plan – 150 minutes (3 periods)

Time	Activity
0 – 10 min	Open the notebook in Colab / your IDE, run the environment check
10 – 30 min	Theory review: loops, lists, // and %, functions (Section 3)
30 – 60 min	Task 1 (weekly flu counts) and Task 2 (H–M–S peeling)
60 – 90 min	Task 3 (wage growth) and Task 4 (perfect numbers)
90 – 120 min	Task 5 (π – Leibniz), Task 6 (Fibonacci), Task 7 (binary → decimal)
120 – 145 min	Extended exercises of your choice + Section 5 exercises (Tasks 8 – 10)
145 – 150 min	Save, export and submit (Section 6)

1. Objectives

After this lab you will be able to:

- Use `for/range()` to iterate over daily inputs, and store inputs in a list to calculate total, average, min, max and rolling mean.
- Use `//` and `%` to decompose quantities (time, money) and extend the logic to other unit conversions.
- Apply the compound growth formula $w = o(1 + p)^n$ with a loop and format tabular output with f-strings.
- Implement divisor summation with a `while` loop and optimize it by iterating only up to $\sqrt{n}$.
- Approximate π with the Leibniz series and analyze convergence.
- Generate the Fibonacci sequence iteratively (tuple assignment, list comprehension).
- Convert binary numbers to decimal using positional weights.
- Write pseudocode before coding, and test programs with known inputs/outputs.

2. Environment Setup

Use any of the environments installed in **Lab 1 (Section 2)**. Quick reference:

Environment	Install / start	Run code
Google Colab (recommended)	https://colab.research.google.com → <i>File</i> → <i>Upload notebook</i> → FoP_Lab2.ipynb → <i>File</i> → <i>Save a copy in Drive</i>	Shift + Enter in a cell; <i>Runtime</i> → <i>Run all</i>
Visual Studio Code	Install Python 3.12+, VS Code, and the <i>Python</i> + <i>Jupyter</i> extensions; select the interpreter with Ctrl+Shift+P → <i>Python: Select Interpreter</i>	 <i>Run Python File</i> , Ctrl+F5, or python file.py in the terminal; notebooks: <i>Run All</i>
PyCharm	<i>New Project</i> → <i>Pure Python</i> (virtualenv, Python 3.12+); packages via <i>Settings</i> → <i>Project</i> → <i>Python Interpreter</i> → +	Right-click → <i>Run 'file'</i> (Ctrl+Shift+F10 / Ctrl+Shift+R on Mac)
JupyterLab	pip install jupyterlab → jupyter lab	Shift + Enter; <i>Kernel</i> → <i>Restart Kernel and Run All Cells</i>
Jupyter Notebook	pip install notebook → jupyter notebook	Shift + Enter; <i>Kernel</i> → <i>Restart & Run All</i>

Packages used in this lab: pandas and matplotlib (only for the extended exercises)

TERMINAL / COMMAND PROMPT [added]

```
# Desktop IDEs (run once in a terminal / inside your virtual environment)
pip install pandas matplotlib
# Google Colab: these packages are pre-installed – no action needed
```

PYTHON [added]

```
# Environment check – run this cell first
import sys
print("Python", sys.version.split()[0])
import pandas as pd, matplotlib; print("pandas", pd.__version__, "| matplotlib",
matplotlib.__version__)
```

3. Theory

3.1 The for loop and range()

for item in sequence: repeats the indented block once for every element of a sequence (list, string, range). range(stop), range(start, stop) and range(start, stop, step) generate integers **up to but not including** stop.

```
PYTHON [added]
for d in range(1, 8):          # 1, 2, ..., 7
    print("Day", d, end=" ")
print()
for ch in "IU":               # a string is a sequence of characters
    print(ch)
```

3.2 The while loop

`while condition:` repeats as long as the condition is `True`. Something inside the loop must eventually make the condition `False`; otherwise the loop never ends (press *Stop* ■ / `Ctrl+C`). `break` leaves the loop immediately; `continue` skips to the next iteration.

```
PYTHON [added]
n = 1
while n * n <= 50:             # smallest n with n² > 50
    n += 1
print("first n with n*n > 50:", n)
```

3.3 Lists and the accumulator pattern

A **list** stores many values: `counts = [19, 22, 25]`. Useful built-ins: `len()`, `sum()`, `min()`, `max()`, `sorted()`; methods `append()`, `index()`. The **accumulator** pattern starts with a neutral value (0 for sums, 1 for products, [] for lists) and updates it inside the loop.

```
PYTHON [added]
values = []
total = 0
for k in range(1, 6):
    values.append(k * k)
    total += k * k
print(values, total, total / len(values))
```

3.4 Quotient and remainder

For integers a and $b > 0$: $a == (a // b) * b + a \% b$. `//` gives how many *whole* b fit in a ; `%` gives what is left. Typical uses: digits of a number (`n % 10`, `n // 10`), even/odd tests, unit conversions, clock arithmetic.

3.5 Functions (a first look)

A **function** packages code under a name; `return` sends a result back to the caller. Type hints (`n: int -> bool`) document the expected types.

```
PYTHON [added]
def square(x: float) -> float:
    return x * x

print(square(3), square(1.5))
```

3.6 From problem to program

1. Understand the input and the expected output (write 1–2 examples by hand).
2. Write **pseudocode** – numbered steps in plain language (START ... END).
3. Translate to Python, one step at a time; run after each step.
4. Test with normal, boundary (0, 1, empty) and invalid inputs.

4. Guided Tasks (worked examples)

Each task below gives the goal, the pseudocode and a working **code example**. Run it, study it, then complete the **extended exercises** in the work areas.

Task 1: Weekly Flu Counts

Goals

Collect flu case counts for 7 days using a loop, compute **total**, **average**, **minimum**, and **maximum**, and print a formatted weekly summary.

Learning objectives

- Use `for/range()` to iterate over daily inputs.
- Store inputs in a list and calculate total, average, min, max, and rolling mean.
- Find the day with the highest and lowest values using `enumerate/index`.
- Extend: use built-in functions in pandas to calculate the statistics.

Pseudocode / steps

1. Initialize or input data: use a sample list or loop input.
2. Validate input (non-negative integer).
3. Compute total, average, min, max.
4. Print results in a formatted table and text summary.
5. Modify the code to compute the rolling mean.

Code example

```
PYTHON
# Example: sample dataset
counts = [19, 22, 25, 18, 20, 24, 21]
total, avg = sum(counts), sum(counts) / len(counts)
print('total:', total, 'avg:', round(avg, 2), 'min:', min(counts), 'max:', max(counts))
```

PYTHON

```
# Optional input loop version
counts = []
for d in range(7):
    val = int(input(f'Enter flu cases for day {d}: '))
    counts.append(val)

print('Total:', sum(counts), 'Average:', round(sum(counts) / len(counts), 2))
```

Worked example – validation, peak day and 3-day rolling mean.

PYTHON [added]

```
counts = [19, 22, 25, 18, 20, 24, 21]
days = ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"]

peak = counts.index(max(counts))
low = counts.index(min(counts))
print(f"{'Day':<5}{'Cases':>6}")
for day, c in zip(days, counts):
    print(f"{'day':<5}{'c':>6}")
print(f"Highest: {days[peak]} ({counts[peak]})   Lowest: {days[low]} ({counts[low]})")

rolling = []
for i in range(2, len(counts)):
    rolling.append(round((counts[i - 2] + counts[i - 1] + counts[i]) / 3, 2))
print("3-day rolling mean:", rolling)
```

PYTHON [added]

```
# Input validation: repeat until the user types a non-negative integer
def read_count(prompt):
    while True:
        text = input(prompt)
        if text.isdigit():          # only digits -> non-negative integer
            return int(text)
        print(" Please enter a non-negative integer.")
```

Extended exercises

1. Use pandas to convert the list into a Series and call `.describe()` to get a statistical summary.
2. Plot the 3-day rolling mean of the counts with matplotlib.
3. Detect anomalies where cases differ from the mean by more than $2 \times$ standard deviation.
4. Compare this week's data with the previous week and compute the percent change.
5. Save results to a CSV file using `csv.writer()`.

STUDENT WORK AREA — Task 1 – extended exercises

Paste your code and a screenshot of the output here.

SCHOLAR DISCUSSION — Notes

This task reflects an early version of time-series data collection and descriptive analytics. Functions like `sum()/len()/min()/max()` parallel vectorized pandas operations, while rolling mean and anomaly detection introduce concepts used in forecasting models (ARIMA, exponential smoothing).

COMMON MISTAKES

- Forgetting to convert `input()` to `int` → concatenation instead of arithmetic.
- Using `index()` when duplicates exist (returns only the first index).
- Not rounding averages or handling division precision properly.

Task 2: Iterative H–M–S Peeling with `//` and `%`**Goals**

Convert a total number of seconds into hours, minutes, and seconds using **floor division** (`//`) and **modulus** (`%`).

Learning objectives

- Use `//` and `%` operators to decompose time into H–M–S.
- Write pseudocode that reflects iterative decomposition logic.
- Understand quotient vs remainder operations.
- Extend the logic to other unit conversions (e.g., milliseconds, dollars/cents).

Theory

Floor division (`//`) gives the integer quotient of a division, while modulus (`%`) gives the remainder. They are often used together to break down quantities like seconds → hours–minutes–seconds. This "peeling" approach is common in modular arithmetic and digital systems.

Pseudocode**PSEUDOCODE**

```
START
SET secs = total number of seconds
COMPUTE hours = secs // 3600
COMPUTE remainder1 = secs % 3600
COMPUTE minutes = remainder1 // 60
COMPUTE seconds = remainder1 % 60
DISPLAY hours, minutes, seconds
END
```

Code example

PYTHON

```
secs = 9876 # total seconds
h = secs // 3600
rem = secs % 3600
m = rem // 60
s = rem % 60
print(f"{h} hours - {m} minutes - {s} seconds")
```

SAMPLE OUTPUT [added]

2 hours - 44 minutes - 36 seconds

Extended exercises

1. Ask the user to input the total seconds.
2. Format as HH:MM:SS using an f-string: `print(f"{h:02d}:{m:02d}:{s:02d}")`.
3. Verify by converting back: `total_check = h*3600 + m*60 + s`.
4. Extend to milliseconds → hours–minutes–seconds–milliseconds.
5. **Peel with a loop:** use the list of units `[("day", 86400), ("hour", 3600), ("minute", 60), ("second", 1)]` and print e.g. 1 day 3 hours 46 minutes 40 seconds for 100000 s.

STUDENT WORK AREA — Task 2 – extended exercises

Paste your code and a screenshot of the output here.

COMMON MISTAKES

- Using `/` instead of `//` → results in float division.
- Forgetting to store the remainder before the next division step.
- Reversing the operation order, leading to incorrect minutes/seconds.

Task 3: Wage Growth per Year (1–10) with 3% Raise

Goals

Simulate a yearly wage increase with a constant growth rate of 3%. Use a for loop to compute wages for 10 years and display the results in a formatted table.

Learning objectives

- Apply the compound growth formula $w = o \times (1 + p)^n$.
- Use for-loop iteration to calculate results over multiple years.
- Format tabular output using f-strings and rounding.

- Understand the relationship between compound rate and exponential growth.

Theory

The compound growth formula models repeated percentage increases over time: $w = o(1 + p)^n$, where o is the initial wage, p the growth rate, and n the year number. This exponential pattern is used in finance, salary projections, and data modeling.

Pseudocode

PSEUDOCODE

```
START
SET o = 10.0      # starting wage
SET p = 0.03      # 3% annual raise
FOR each year n from 1 to 10:
    COMPUTE w = o * (1 + p)^n
    PRINT n, w rounded to 2 decimals
END
```

Python code

PYTHON

```
o, p = 10.0, 0.03

print(f"{'Year':<6}{'Wage ($/hr)':>12}")
for n in range(1, 11):
    w = o * (1 + p) ** n
    print(f"{n:<6}{round(w, 2):>12}")
```

SAMPLE OUTPUT

Year	Wage (\$/hr)
1	10.3
2	10.61
3	10.93
4	11.26
5	11.59
6	11.94
7	12.3
8	12.67
9	13.05
10	13.44

Correction: the sample output now matches the program exactly (years 4 and 10 are 11.26 and 13.44). Tip: `f"{w:>12.2f}"` prints 10.30 instead of 10.3.

Extended exercises

1. Allow user input for o , p , and the number of years.
2. Add a column showing the increase each year (difference from the previous year).
3. Plot the wage growth curve using `matplotlib`.
4. Compare multiple rates (2%, 5%, 8%) on the same graph.
5. Adjust for inflation by subtracting an annual 2% rate to compute the real wage.

STUDENT WORK AREA — Task 3 – extended exercises

Paste your code and a screenshot of the output here.

COMMON MISTAKES

- Using `w = o * (1 + p * n)` instead of exponentiation.
- Forgetting to `round()` results to two decimals.
- Incorrect range boundaries (should be `range(1, 11)`).

Task 4: Perfect Number Detector

Goals

Write a program to check whether a number is a perfect number. Reinforce the use of while loops, modulus (%), and paired divisors (`n // i`). Understand the relationship between divisors and sum-of-divisors logic.

NOTE

A perfect number is a positive integer that equals the sum of its proper divisors. For example, $6 = 1 + 2 + 3$ and $28 = 1 + 2 + 4 + 7 + 14$. To find divisors efficiently, only iterate up to $\sqrt{n}$ and add both `i` and `n // i` when `n % i == 0`.

Learning objectives

- Understand the definition of a perfect number (sum of its proper divisors equals itself).
- Implement divisor summation using a while loop and conditional checks.
- Apply optimization by iterating only up to $\sqrt{n}$.
- Test the function on sample numbers [6, 28, 12].

Pseudocode

```
PSEUDOCODE
START
FUNCTION is_perfect(n)
  IF n < 2 THEN RETURN False
  SET s = 1
  SET i = 2
  WHILE i * i <= n DO
    IF n mod i == 0 THEN
      ADD i to s
      IF i != n / i THEN ADD (n / i) to s
    INCREMENT i
  END WHILE
  RETURN (s == n)
END FUNCTION

FOR each n in [6, 28, 12]:
  PRINT n, is_perfect(n)
END
```

Code example

```
PYTHON
def is_perfect(n: int) -> bool:
    if n < 2:
        return False
    s = 1
    i = 2
    while i * i <= n:
        if n % i == 0:
            s += i
            if i != n // i:
                s += n // i
        i += 1
    return s == n

for n in [6, 28, 12]:
    print(n, is_perfect(n))
```

SAMPLE OUTPUT

```
6 True
28 True
12 False
```

Extended exercises

- Ask the user to input a number and test it using `is_perfect()`.
- List all perfect numbers between 1 and 10000.
- Count how many perfect numbers exist in a given range.
- Add an iteration counter to display how many divisors were checked.
- Plot the distribution of perfect numbers using `matplotlib`.

STUDENT WORK AREA — Task 4 – extended exercises

Paste your code and a screenshot of the output here.

COMMON MISTAKES

- Forgetting to initialize `s = 1` (since 1 divides every `n`).
- Failing to check `i != n // i` before adding mirrored divisors.
- Using `i <= n` instead of `i * i <= n`, causing redundant computation.
- Returning an integer instead of a Boolean value.

Task 5: π Approximation (Leibniz Series)

Goals

Approximate the value of π using the Leibniz series. Implement a for loop to accumulate alternating terms and observe the convergence behavior as the number of terms increases.

Learning objectives

- Understand the Leibniz formula for π approximation.
- Implement alternating summation using a for loop.
- Compute the absolute error against `math.pi`.
- Analyze the convergence rate as the number of terms increases.

Theory

The Leibniz series provides a simple alternating series for π approximation:

$$\pi = 4 \times (1 - 1/3 + 1/5 - 1/7 + 1/9 - \dots)$$

Each iteration alternates the sign and adds a new fractional term. Although it converges slowly, it is conceptually important for understanding numerical convergence.

Pseudocode

PSEUDOCODE

START

FUNCTION pi_leibniz(k_terms)

 SET sgn = 1

 SET acc = 0

 FOR k = 0 TO k_terms - 1

 acc = acc + sgn / (2*k + 1)

 sgn = sgn * -1

 END FOR

 RETURN 4 * acc

END FUNCTION

FOR each t in [100, 1000, 10000]:

 COMPUTE approx = pi_leibniz(t)

 COMPUTE error = abs(math.pi - approx)

 PRINT t, approx, error

END

Python code

PYTHON

```
import math
```

```
def pi_leibniz(k_terms: int) -> float:
```

```
    sgn = 1.0
```

```
    acc = 0.0
```

```
    for k in range(k_terms):
```

```
        acc += sgn / (2 * k + 1)
```

```
        sgn *= -1
```

```
    return 4 * acc
```

```
for t in [100, 1000, 10000]:
```

```
    approx = pi_leibniz(t)
```

```
    print(f"terms={t}<6> π≈{approx:.6f} error={abs(math.pi - approx):.6f}")
```

SAMPLE OUTPUT

```
terms=100    π≈3.131593 error=0.010000
```

```
terms=1000   π≈3.140593 error=0.001000
```

```
terms=10000  π≈3.141493 error=0.000100
```

Extended exercises

1. Allow user input for an error threshold ϵ and stop when `abs(math.pi - 4*acc) <  $\epsilon$` .
2. Plot the convergence curve (terms vs error) using matplotlib (log scale).
3. Compare with faster series (Nilakantha, Ramanujan).
4. Visualize the alternating signs (+1/1, -1/3, +1/5, ...) with a bar chart.

STUDENT WORK AREA — Task 5 – extended exercises

Paste your code and a screenshot of the output here.

COMMON MISTAKES

- Using `range(1, k_terms+1)` but miscalculating the term indices (off by one).
- Forgetting to multiply the result by 4.
- Not flipping the sign correctly each iteration.
- Using `int` instead of `float`, causing precision errors.

Task 6: Fibonacci nth Term and Sequence

Goals

Generate the Fibonacci sequence and find the nth Fibonacci number. Reinforce the use of for loops and tuple assignment, and compare iterative vs recursive logic.

Learning objectives

- Understand the Fibonacci recurrence relation $F_n = F_{n-1} + F_{n-2}$.
- Implement Fibonacci generation using an iterative for loop.
- Use a list comprehension to produce a sequence of terms.
- Explore growth patterns, ratios, and visualization extensions.

Theory

The Fibonacci sequence starts with $F_0 = 0$, $F_1 = 1$, and each next term equals the sum of the two preceding terms. This sequence grows exponentially and appears in nature, finance, and AI pattern modeling. Iterative computation avoids recursion overhead and allows $O(n)$ efficiency.

Pseudocode

PSEUDOCODE

START

FUNCTION fib(n)

 SET a = 0, b = 1

 FOR i = 0 TO n-1:

 SET temp = a

 a = b

 b = temp + a

 RETURN a

END FUNCTION

CREATE list seq = []

FOR i = 0 TO 9:

 APPEND fib(i) TO seq

PRINT seq

END

Python code

PYTHON

```
def fib(n: int) -> int:
```

```
    a, b = 0, 1
```

```
    for _ in range(n):
```

```
        a, b = b, a + b
```

```
    return a
```

```
seq = [fib(i) for i in range(10)]
```

```
print(seq)
```

SAMPLE OUTPUT

```
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
```

Extended exercises

- Allow user input: ask for n and print F(n).
- Display the ratios $F(n)/F(n-1)$ to explore convergence to the Golden Ratio.
- Plot Fibonacci growth using matplotlib (x-axis: n, y-axis: F(n)).
- Compare iterative and recursive implementations and measure their speed.
- Apply modulo operations for pattern- or cryptography-based tasks.

STUDENT WORK AREA — Task 6 – extended exercises

Paste your code and a screenshot of the output here.

SCHOLAR DISCUSSION

The Fibonacci sequence exemplifies linear recursion and exponential growth. The ratio $F(n+1)/F(n)$ approaches the Golden Ratio $\phi \approx 1.618$, seen in natural and financial systems. The iterative implementation ($O(n)$) is efficient compared to the naive recursive one ($O(2^n)$), and the logic aligns with predictive models in time-series and machine learning.

COMMON MISTAKES

- Reversing the update order of `a` and `b`.
- Using recursion without base cases (causes infinite recursion).
- Incorrect range in the list comprehension (missing the first element).
- Forgetting to initialize `a, b = 0, 1` properly.

Task 7: Binary to Decimal (Positional Weights)**Goals**

Understand how to convert binary numbers to decimal using positional weights. Practice looping through string bits and accumulating positional values.

Learning objectives

- Explain the concept of positional weights in binary representation.
- Implement binary-to-decimal conversion using iterative accumulation.
- Compare manual computation with Python's built-in `int(bits, 2)`.
- Extend the concept to other numeral systems (octal, hexadecimal).

Theory

In a binary number, each bit has a positional weight that is a power of two. For example, $1101_2 = 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 = 13_{10}$. The iterative approach multiplies the accumulated value by 2 and adds 1 if the current bit is `'1'`.

Pseudocode

```
PSEUDOCODE
START
FUNCTION bin_to_dec(bits)
  SET val = 0
  FOR each character ch in bits:
    IF ch == '1' THEN
      val = val * 2 + 1
    ELSE
      val = val * 2 + 0
  END FOR
  RETURN val
END FUNCTION

PRINT bin_to_dec('1101')
END
```

Python code

```
PYTHON
def bin_to_dec(bits: str) -> int:
    val = 0
    for ch in bits:
        val = val * 2 + (1 if ch == '1' else 0)
    return val

print(bin_to_dec('1101'))
```

SAMPLE OUTPUT

13

Extended exercises

- Allow user input and validate the binary string.
- Compare the result with `int(bits, 2)`.
- Implement the reverse conversion (decimal to binary).
- Handle invalid input gracefully (non-binary characters).
- Plot bit positions and powers of 2 using matplotlib.

STUDENT WORK AREA — Task 7 – extended exercises

Paste your code and a screenshot of the output here.

COMMON MISTAKES

- Processing bits in reverse order (right-to-left) incorrectly.
- Forgetting to multiply by 2 before adding the current bit.
- Not checking for invalid characters (anything other than '0' or '1').
- Assuming the input is an integer instead of a string.

5. Exercises

Solve these without looking at a solution first. For each task: (1) write the pseudocode, (2) write the program, (3) run it with the sample input and with your own test cases.

Task 8 – Prime number checker and prime list

Write a function `is_prime(n)` that returns `True` if `n` is prime. Use a `while` loop that tests divisors only up to $\sqrt{n}$ (as in Task 4). Then print all primes below 100, ten per line, right-aligned in width 4, and the number of primes found.

SAMPLE OUTPUT [added]

```
 2  3  5  7 11 13 17 19 23 29
31 37 41 43 47 53 59 61 67 71
73 79 83 89 97
25 primes below 100
```

STARTER CODE (PYTHON) [added]

```
def is_prime(n: int) -> bool:
    if n < 2:
        return False
    i = 2
    while i * i <= n:
        # TODO: return False if i divides n
        i += 1
    return True

count = 0
for n in range(2, 100):
    if is_prime(n):
        # TODO: print n in width 4 without a newline; start a new line after 10 primes
        count += 1
print()
print(count, "primes below 100")
```

STUDENT WORK AREA — Task 8 — your solution

Paste your code and a screenshot of the output here.

Task 9 – GCD and LCM with Euclid's algorithm

The greatest common divisor satisfies $\text{gcd}(a, b) = \text{gcd}(b, a \% b)$ and $\text{gcd}(a, 0) = a$. Write `gcd(a, b)` with a `while` loop (no `math.gcd`), print every step, and compute $\text{lcm}(a, b) = a \times b // \text{gcd}(a, b)$. Check your result with `math.gcd`.

SAMPLE OUTPUT [added]

```
a = 84, b = 36
84 % 36 = 12
36 % 12 = 0
gcd = 12, lcm = 252
```

STUDENT WORK AREA – Task 9 – your solution

Paste your code and a screenshot of the output here.

Task 10 – Collatz sequence (while loop)

Start from a positive integer n . If n is even, $n \leftarrow n // 2$; otherwise $n \leftarrow 3n + 1$. Repeat until $n = 1$. Print the sequence and the number of steps. Then find the starting value below 1000 with the longest sequence.

SAMPLE OUTPUT [added]

```
n = 6: 6 3 10 5 16 8 4 2 1 (8 steps)
Longest below 1000: n = 871 with 178 steps
```

STUDENT WORK AREA – Task 10 – your solution

Paste your code and a screenshot of the output here.

6. Submission Instructions

What to submit

1. **Source code folder (/codes/)** – each task should be implemented in a separate `.py` file:

```
codes/
├── task1_weekly_flu.py
├── task2_hms_peeling.py
├── task3_wage_growth.py
├── task4_perfect_number.py
└── ...
```

Each file must contain a header comment with student info:

```
PYTHON
# Student Name:
# Student ID:
# Course: Fundamentals of Programming (FoP)
# Lab 2 – Task X
# Date:
# Description: Short summary of the task
```

Ensure each script runs correctly and includes inline comments for clarity.

2. **Output evidence folder (/outputs/)** – include screenshots of successful runs for each task (task1_output.png ... task7_output.png). Screenshots must clearly show the input, output, and any charts produced.
3. **Lab report (FoP_Lab2_Report.docx or .pdf)** – your report must include: cover page (name, student ID, course, semester, submission date); table of contents; Tasks 1–7 (and 8–10); summary: what you learned.
4. **Submission format** – the final submission is one folder (or ZIP) named IT149IU_Lab2_FullName_StudentID containing codes/, outputs/, FoP_Lab2_Report.docx and README.txt.

How to submit (Google Form)

All labs are submitted through **one shared Google Form**. Submit a **link** to a folder or repository that contains your work.

Step 1 – Organize your work in one folder named IT149IU_Lab2_FullName_StudentID (e.g. IT149IU_Lab2_NguyenVanAn_ITITI24001):

```
IT149IU_Lab2_FullName_StudentID/
├── FoP_Lab2_FullName_StudentID.ipynb
├── codes/
│   ├── Task1.py
│   └── ...
├── outputs/
│   ├── task1_output.png
│   └── ...
├── FoP_Lab2_Report.pdf
└── README.md
```

- the **notebook** – completed, with all cells executed;
- **codes/** – one .py file per task, each starting with the header comment;
- **outputs/** – screenshots that clearly show input and output;
- the **lab report** (.docx or .pdf) and a **README.md** (name, student ID, how to run, notes).

Step 2 – Upload the folder using ONE of these options:

1. **GitHub (recommended)**: create a repository (e.g. IT149IU-FoP-Labs, one sub-folder per lab), push your files and make the repository *Public* (or add the instructor as a collaborator).

2. **Google Drive:** upload the folder → right-click → *Share* → *General access: Anyone with the link – Viewer* → *Copy link*.
3. **Google Colab only:** *Share* → *Anyone with the link – Viewer* → *Copy link* (use this only if all work is inside the notebook).

TERMINAL / COMMAND PROMPT [added]

```
# GitHub from the command line (first time)
git init
git add .
git commit -m "Lab 2 - FullName - StudentID"
git branch -M main
git remote add origin https://github.com/<your-username>/IT149IU-FoP-Labs.git
git push -u origin main
# later labs: git add . ; git commit -m "Lab N" ; git push
```

Step 3 – Submit the link in the Google Form

Item	Details
Submission form for Lab 2 (Lab number pre-selected)	Open the Lab 2 submission form
Short link (choose "Lab 2" in the form)	https://forms.gle/aHqDLDZ6YUyzoYVv8
Fields to fill in	Full name · Student ID · Class/Group · Lab number · Submission type · Submission link
Deadline	End of the lab week – Sunday 23:59 (unless the instructor announces another date)

Before you press Submit – checklist

- The notebook runs from top to bottom without errors (*Runtime* → *Restart session and run all*).
- Every task has code, output (screenshot or cell output) and, where requested, pseudocode.
- Each `.py` file starts with the header comment (name, ID, course, lab, task, date, description).
- The link opens in a private/incognito browser window (sharing permission is correct).
- You may re-submit (edit your response) before the deadline; the last submission is graded.

NOTE — Grading guide (per lab)

Criterion	Weight
Correctness of programs (all required tasks)	50%
Pseudocode / algorithm and explanation	15%
Code quality (names, comments, functions, input validation)	15%
Output evidence (screenshots / executed notebook)	10%
Extensions and challenge tasks	10%

COMMON MISTAKES — Academic integrity

Discuss ideas with classmates, but write your own code. Copied submissions receive zero for all students involved. If you use an AI assistant, state which tool and for which part in the *Notes* field of the form and in your README.